

Getting Started with Pelee and Rondeau

Fluids Lab

October 3, 2026

Contents

1	Introduction	6
1.1	Why Are We Using Servers?	6
1.2	Important Concepts	6
1.3	Learning Objectives	6
2	Connecting to a Server	7
2.1	What is SSH?	7
2.2	Windows	7
2.3	macOS and Linux	7
2.4	First Login	7
2.5	Successful Login	7
2.6	Logging Out	7
2.7	Exercise	7
3	Linux Navigation	8
3.1	Where Am I?	8
3.2	Listing Files	8
3.3	Changing Directories	8
3.4	Creating Directories	8
3.5	Useful Shortcuts	8
3.6	Exercise	8
4	Files and Directories	9
4.1	Creating Files	9
4.2	Copying Files	9
4.3	Moving and Renaming Files	9
4.4	Deleting Files	9
4.5	Working with Directories	9
4.6	Exercise	9
5	Editing Files	10
5.1	Using Nano	10
5.2	Saving Changes	10
5.3	Exiting	10
5.4	Viewing a File	10
5.5	Exercise	10

6	Running Programs	11
6.1	Python	11
6.2	Julia	11
6.3	Useful Commands	11
7	Conda Environments	12
7.1	Creating an Environment	12
7.2	Activating an Environment	12
7.3	Installing Packages	12
7.4	Leaving an Environment	12
8	Environment Modules	13
8.1	Listing Available Modules	13
8.2	Loading a Module	13
8.3	Checking Loaded Modules	13
8.4	Unloading a Module	13
8.5	Common Examples	14
8.6	Finding Software	14
8.7	Exercise	14
8.8	A Common Mistake	14
9	Python Virtual Environments	15
9.1	Creating a Virtual Environment	15
9.2	Activating the Environment	15
9.3	Installing Packages	15
9.4	Leaving the Environment	15
9.5	When Should I Use Conda?	15
9.6	When Should I Use a Virtual Environment?	16
9.7	Exercise	16
9.8	Recommended Practice	16
10	Long-Running Jobs	17
10.1	Using tmux	17
10.2	Detach	17
10.3	Reconnect	17
10.4	Exercise	17
11	Monitoring Resources	18
11.1	CPU and Memory	18
11.2	Disk Usage	18
11.3	Exercise	18
12	GPU Computing	19
12.1	Checking GPU Usage	19
12.2	Good Practice	19
12.3	Exercise	19

13 Transferring Files	20
13.1 SCP	20
13.2 RSYNC	20
13.3 Exercise	20
14 Git Basics	21
14.1 Cloning a Repository	21
14.2 Saving Changes	21
14.3 Uploading Changes	21
14.4 Getting Updates	21
14.5 Exercise	21
15 Good Computing Citizenship	22
15.1 Storage	22
15.2 File Naming	22
16 Reproducible Research	23
16.1 Recommended Project Structure	23
16.2 Best Practices	23
16.3 Exercise	23
17 Troubleshooting	24
17.1 Command Not Found	24
17.2 Disk Full	24
17.3 Permission Denied	24
17.4 Program Crashed	24
17.5 Before Asking for Help	24
A Vim Survival Guide	25
A.1 Insert Mode	25
A.2 Return to Command Mode	25
A.3 Common Commands	25
A.4 Most Important Command	25
B File Permissions	25
B.1 Making a Script Executable	26
B.2 Useful Commands	26
C SSH Keys	27
C.1 Create a Key Pair	27
C.2 Install the Public Key	27
C.3 Test	27
D VSCode Remote SSH	28
D.1 Install	28
D.2 Connect	28
D.3 Benefits	28

E	Jupyter Notebooks	29
E.1	Start Jupyter	29
E.2	Create an SSH Tunnel	29
E.3	Open in a Browser	29
F	Alliance Clusters	30
G	Pelee and Rondeau Cheat Sheet	31
H	Multi-GPU Computing with Julia	32
H.1	Loading Required Modules	32
H.2	Installing Required Julia Packages	32
H.3	Testing CUDA	33
H.4	Configuring MPI.jl	33
H.5	Verifying GPU Assignment	33
H.6	Selecting GPUs	34
H.7	Checking Available GPUs	34
H.8	Checking Which GPU Julia Is Using	34
I	Multi-GPU Computing with Python	35
I.1	Loading Required Modules	35
I.2	Installing Required Packages	35
I.3	Testing CUDA	35
I.4	Verifying GPU Assignment	36
I.5	Selecting GPUs	36
I.6	MPI with Python	36
I.7	Multi-GPU PyTorch	37
I.8	Troubleshooting	37
J	Multi-GPU Oceananigans on Nibi (Quick Start)	38
J.1	Login	38
J.2	Load Modules	38
J.3	Move the Julia Depot to Scratch	38
J.4	Clone Oceananigans	38
J.5	Instantiate Oceananigans	39
J.6	Instantiate the Benchmarking Environment	39
J.7	Request an Interactive H100 Node	39
J.8	Verify the GPU	39
J.9	Verify CUDA	40
J.10	Configure MPI.jl to Use System OpenMPI	40
J.11	Verify CUDA-Aware MPI	40
J.12	Simple MPI+CUDA Test	41
J.13	Run a Benchmark	41
J.14	Notes	42

K	Contributing to This Handbook	43
K.1	Repository	43
K.2	Reporting Errors	43
K.3	Suggesting Improvements	43
K.4	Submitting Changes	43
K.5	Source Files	43

1 Introduction

1.1 Why Are We Using Servers?

Most scientific models and datasets are too large to run efficiently on a laptop. The Fluids Lab servers, **Pelee** and **Rondeau**, provide access to faster CPUs, GPUs, larger memory, and shared storage.

You may think of them as powerful Linux computers that you access remotely.

Your Laptop $\longleftrightarrow$ Pelee / Rondeau

You work from your laptop, but the calculations happen on the server.

1.2 Important Concepts

Multi-user systems. Many researchers share the same server.

CPU versus GPU. A CPU is designed for general-purpose computing. A GPU is designed for highly parallel computations.

Home directory. Every user has a home directory:

`/home/username`

Note: for historical reasons at UWaterloo Math, `/home` is an alias for `/u`, although they have the same purpose and meaning.

Persistence. Files remain on the server after you log out.

1.3 Learning Objectives

- connect to Pelee and Rondeau;
- navigate the Linux filesystem;
- transfer files;
- run Python and Julia programs;
- monitor resource usage.

2 Connecting to a Server

2.1 What is SSH?

SSH (Secure Shell) is the standard method for connecting to a remote Linux machine.

```
ssh username@pelee.math.private.uwaterloo.ca
```

```
ssh username@rondeau.math.private.uwaterloo.ca
```

2.2 Windows

Open Windows Terminal and run the SSH command.

2.3 macOS and Linux

Open a terminal and run the SSH command.

Note: Nothing appears on the screen while typing your password. This is normal.

2.4 First Login

Type **yes** when asked whether you trust the host.

2.5 Successful Login

```
username@pelee:~$
```

2.6 Logging Out

```
exit
```

2.7 Exercise

1. Connect to Pelee.
2. Run `pwd`.
3. Disconnect using `exit`.

3 Linux Navigation

Most interactions with a Linux server occur through a command-line interface.

3.1 Where Am I?

```
pwd
```

Example:

```
/home/username
```

3.2 Listing Files

```
ls
ls -l
ls -lh
```

3.3 Changing Directories

```
cd project1
cd ..
cd
```

Typing `cd` without an argument returns you to your home directory.

3.4 Creating Directories

```
mkdir project1
mkdir code data figures
```

3.5 Useful Shortcuts

Symbol	Meaning
.	Current directory
..	Parent directory
~	Home directory

3.6 Exercise

Create:

Try the following commands:

```
mkdir research
cd research
mkdir code data figures
ls
```

4 Files and Directories

Files and directories are the basic building blocks of a Linux filesystem.

4.1 Creating Files

Create an empty file:

```
touch notes.txt
```

Verify that it exists:

```
ls
```

4.2 Copying Files

Copy a file:

```
cp notes.txt backup.txt
```

The original file is unchanged.

4.3 Moving and Renaming Files

Rename a file:

```
mv notes.txt notes_old.txt
```

Move a file into another directory:

```
mv notes_old.txt data/
```

4.4 Deleting Files

Delete a file:

```
rm notes.txt
```

Warning: Deleted files cannot usually be recovered.

4.5 Working with Directories

Copy a directory and its contents:

```
cp -r project project_backup
```

Delete a directory and its contents:

```
rm -r project_backup
```

4.6 Exercise

Create a file called `notes.txt`. Copy it, rename it, and then delete the copy.

5 Editing Files

Many scientific workflows require editing scripts, configuration files, and text files.

5.1 Using Nano

Nano is a simple text editor that is available on most Linux systems.

Open a file:

```
nano notes.txt
```

If the file does not exist, it will be created.

5.2 Saving Changes

To save:

```
Ctrl+O
```

Press Enter to confirm.

5.3 Exiting

To exit:

```
Ctrl+X
```

5.4 Viewing a File

Display a file:

```
cat notes.txt
```

For longer files:

```
less notes.txt
```

Press q to quit.

5.5 Exercise

Create a file called `todo.txt`. Add three lines of text using Nano.

6 Running Programs

Most users connect to Pelee and Rondeau in order to run scientific software.

6.1 Python

Example file, `hello.py`, with contents:

```
print("Hello World")
```

Run using:

```
python hello.py
```

6.2 Julia

Example file, `hello.jl`, with contents:

```
println("Hello World")
```

Run using:

```
julia hello.jl
```

6.3 Useful Commands

```
which python  
echo $PATH
```

7 Conda Environments

Different projects often require different software versions.

7.1 Creating an Environment

```
conda create -n ocean python=3.12
```

7.2 Activating an Environment

```
conda activate ocean
```

7.3 Installing Packages

```
conda install numpy
```

7.4 Leaving an Environment

```
conda deactivate
```

8 Environment Modules

Many software packages on Pelee and Rondeau are managed using *environment modules*.

Modules allow users to select software versions without manually modifying environment variables.

8.1 Listing Available Modules

Display all available modules:

```
module avail
```

Example software available through modules includes

- CUDA
- Julia
- MATLAB
- Anaconda
- OpenMPI
- ParaView

8.2 Loading a Module

Load Julia:

```
module load julia
```

or load a specific version:

```
module load julia/1.10.8
```

8.3 Checking Loaded Modules

Show loaded modules:

```
module list
```

8.4 Unloading a Module

Unload Julia:

```
module unload julia
```

Unload all modules:

```
module purge
```

8.5 Common Examples

Load CUDA:

```
module load cuda/12.3.2
```

Load OpenMPI:

```
module load openmpi/4.1.7-with-cuda
```

Load Julia:

```
module load julia/1.10.8
```

8.6 Finding Software

Search for a package:

```
module spider julia
```

or

```
module spider cuda
```

8.7 Exercise

Load Julia and verify the version:

```
module load julia/1.10.8  
julia --version
```

8.8 A Common Mistake

Modules are usually reset when you start a new login session. If a command suddenly disappears after logging in, check whether the required module has been loaded:

```
module list
```

9 Python Virtual Environments

Python includes a built-in mechanism for creating isolated environments.

Virtual environments provide many of the same benefits as Conda and are often simpler for Python-only projects.

9.1 Creating a Virtual Environment

Create a new environment called `venv`:

```
python -m venv venv
```

This creates a directory containing an isolated Python installation.

9.2 Activating the Environment

Activate the environment:

```
source venv/bin/activate
```

Your command prompt should change to indicate that the environment is active.

9.3 Installing Packages

Use `pip` to install packages:

```
pip install numpy
```

You may verify the installation:

```
python -c "import numpy"
```

9.4 Leaving the Environment

Deactivate the environment:

```
deactivate
```

9.5 When Should I Use Conda?

Conda is often easier when working with:

- JupyterLab
- CUDA-enabled Python packages
- packages with compiled dependencies
- mixed Python and non-Python software

9.6 When Should I Use a Virtual Environment?

Virtual environments are often a good choice for:

- lightweight Python projects
- software development
- projects using only pip-installed packages

9.7 Exercise

Create a virtual environment, install NumPy, verify that it imports successfully, and then deactivate the environment.

9.8 Recommended Practice

Create a separate environment for each project.

For example:

```
arctic_project_env  
oceananigans_env  
machine_learning_env
```

Avoid installing large numbers of unrelated packages into a single environment.

10 Long-Running Jobs

Many computations run for hours or days. If your SSH connection is interrupted, the job may stop.

10.1 Using tmux

Start a tmux session:

```
tmux
```

Run your program normally.

10.2 Detach

Leave the session running:

```
Ctrl+b d
```

10.3 Reconnect

List sessions:

```
tmux ls
```

Reconnect:

```
tmux attach
```

10.4 Exercise

Start tmux, run a simple program, detach, and reconnect.

11 Monitoring Resources

Before running large jobs, monitor resource usage.

11.1 CPU and Memory

```
htop
```

or

```
top
```

11.2 Disk Usage

Available space:

```
df -h
```

Directory sizes:

```
du -sh *
```

11.3 Exercise

Determine how much free disk space is available and identify the largest directory in your project.

12 GPU Computing

Many scientific applications can use GPUs.

12.1 Checking GPU Usage

```
nvidia-smi
```

This displays:

- GPU memory usage;
- GPU utilization;
- running processes.

12.2 Good Practice

Always check whether a GPU is busy before starting large jobs.

12.3 Exercise

Run `nvidia-smi` and identify the available GPUs.

13 Transferring Files

Move files between your laptop and the server using SCP or rsync.

13.1 SCP

```
scp file.txt username@pelee.math.private.uwaterloo.ca:~
```

13.2 RSYNC

```
rsync -av data/ username@pelee.math.private.uwaterloo.ca:data/
```

13.3 Exercise

Transfer a small text file and verify the transfer using `ls`.

14 Git Basics

Git is a version control system that helps track changes to code and documents.

14.1 Cloning a Repository

```
git clone https://github.com/user/repo.git
```

14.2 Saving Changes

```
git add file.py  
git commit -m "Update analysis"
```

14.3 Uploading Changes

```
git push
```

14.4 Getting Updates

```
git pull
```

14.5 Exercise

Create a new directory called `git-practice`:

```
mkdir git-practice  
cd git-practice
```

Initialize a Git repository:

```
git init
```

Create a file:

```
echo "Hello Git" > README.txt
```

Add the file:

```
git add README.txt
```

Commit the file:

```
git commit -m "Initial commit"
```

Verify the commit history:

```
git log --oneline
```

15 Good Computing Citizenship

Pelee and Rondeau are shared resources.

- Keep directories organized.
- Delete unnecessary files.
- Do not monopolize CPUs or GPUs.
- Use descriptive file names.
- Document your work.

15.1 Storage

Home directories should only be used for storing small files like text files or code. Large data, such as binary output datasets, should be stored on the large RAID (Redundant Array of Inexpensive Disks) volumes, named 'fsys1' and 'fsys2'. Your folder on these storage volumes can be accessed on either pelee or rondeau by cd'ing to:

```
/mnt/autofs/sutton.math/fsys1/username  
/mnt/autofs/sutton.math/fsys2/username
```

Replace 'username' in the above with your UWaterloo 8-character userid.

15.2 File Naming

Avoid:

```
test  
test2  
test_final  
test_final2
```

Prefer:

```
arctic_run_2026_09_18
```

16 Reproducible Research

Good research should be easy for others (and your future self) to reproduce.

16.1 Recommended Project Structure

```
project/  
|-- code/  
|-- data/  
|-- figures/  
|-- notebooks/  
'-- manuscript/
```

16.2 Best Practices

- Use Git.
- Use Conda environments.
- Keep raw data separate from results.
- Write a README file.

16.3 Exercise

Create a project directory using the structure above.

17 Troubleshooting

17.1 Command Not Found

```
which python
```

Verify that the program is installed and available in your PATH.

17.2 Disk Full

```
df -h
```

17.3 Permission Denied

```
ls -l
```

Check file ownership and permissions.

17.4 Program Crashed

Check:

- available memory;
- available disk space;
- error messages.

17.5 Before Asking for Help

Include:

- the command you ran;
- the full error message;
- what you have already tried.

A Vim Survival Guide

Sooner or later you will encounter Vim.

Open a file:

```
vim file.txt
```

A.1 Insert Mode

Press

```
i
```

to begin editing.

A.2 Return to Command Mode

Press

```
Esc
```

A.3 Common Commands

Save:

```
:w
```

Quit:

```
:q
```

Save and quit:

```
:wq
```

Quit without saving:

```
:q!
```

A.4 Most Important Command

If you become stuck:

```
Esc :q!
```

B File Permissions

Linux controls who may read, write, and execute files.

Display permissions:

```
ls -l
```

Example:

```
-rwxr-xr-x script.sh
```

B.1 Making a Script Executable

```
chmod u+x script.sh
```

You can then run

```
./script.sh
```

B.2 Useful Commands

Give read and write permission to the owner:

```
chmod 600 file.txt
```

Make a file executable:

```
chmod 755 script.sh
```

C SSH Keys

SSH keys allow password-free login.

C.1 Create a Key Pair

```
ssh-keygen
```

Accept the default location.

C.2 Install the Public Key

```
ssh-copy-id username@pelee.math.private.uwaterloo.ca
```

C.3 Test

```
ssh username@pelee.math.private.uwaterloo.ca
```

You should no longer be asked for a password.

D VSCode Remote SSH

Many users prefer VSCode instead of terminal editors.

D.1 Install

Install

- Visual Studio Code
- Remote SSH Extension

D.2 Connect

Open the command palette:

Ctrl+Shift+P

Select

Remote-SSH: Connect to Host

and connect to Pelee or Rondeau.

D.3 Benefits

- graphical file browser;
- interactive terminal;
- syntax highlighting;
- Git integration.

E Jupyter Notebooks

Jupyter allows interactive computing through a web browser.

E.1 Start Jupyter

```
jupyter lab --no-browser
```

The server will display a URL and port number.

E.2 Create an SSH Tunnel

From your laptop:

```
ssh -L 8888:localhost:8888 username@pelee
```

E.3 Open in a Browser

Open

```
http://localhost:8888
```

in your browser.

F Alliance Clusters

The skills learned on Pelee and Rondeau transfer directly to national HPC systems.

Examples:

- Narval
- Béluga
- Cedar
- Graham

Daily tasks remain similar:

```
ssh cd ls tmux git conda
```

The major difference is that most large jobs must be submitted through a scheduler.

G Pelee and Rondeau Cheat Sheet

Task	Command
Login	ssh username@pelee
Current directory	pwd
List files	ls -lh
Create directory	mkdir project
Edit file	nano file.txt
Copy file	cp old new
Move file	mv old new
Delete file	rm file.txt
Start tmux	tmux
Reconnect tmux	tmux attach
Disk usage	df -h
Directory size	du -sh *
Check GPUs	nvidia-smi
Logout	exit

H Multi-GPU Computing with Julia

Many Julia applications can use multiple GPUs simultaneously. Typical examples include:

- Oceananigans.jl
- MPI-based PDE solvers
- Distributed machine learning
- Large ensemble simulations

Multi-GPU programs typically require a CUDA-aware MPI installation. The typical workflow is:

1. Load CUDA, OpenMPI, and Julia modules.
2. Configure MPI.jl to use the system MPI installation.
3. Launch Julia using `mpiexec`.
4. Verify that each MPI rank can access a GPU.

H.1 Loading Required Modules

Load the required software:

```
module load cuda/12.3.2
module load openmpi/4.1.7-with-cuda
module load julia/1.10.8
```

Verify that the modules are loaded:

```
module list
```

Verify CUDA:

```
nvidia-smi
```

Verify MPI:

```
which mpiexec
```

H.2 Installing Required Julia Packages

The examples in this appendix use the following Julia packages:

- CUDA.jl
- MPI.jl
- MPIPreferences.jl

If they are not already installed, start Julia and run:

```
using Pkg
Pkg.add("CUDA")
Pkg.add("MPI")
Pkg.add("MPIPreferences")
Pkg.add("Oceananigans")
```

H.3 Testing CUDA

Before testing MPI, verify that Julia can see a CUDA-enabled GPU. Start Julia:

```
julia
```

Load CUDA.jl and check for GPUs:

```
using CUDA
CUDA.has_cuda_gpu()
```

Expected output:

```
true
```

You may also display information about the available GPUs:

```
CUDA.devices()
```

H.4 Configuring MPI.jl

Inside Julia:

```
using MPIPreferences
MPIPreferences.use_system_binary()
```

Restart Julia and verify:

```
using MPI
MPI.versioninfo()
```

H.5 Verifying GPU Assignment

Create a file called `test.jl`:

```
using MPI
using CUDA
MPI.Init()
rank = MPI.Comm_rank(MPI.COMM_WORLD)
println( "Rank $rank using GPU $(CUDA.device())" )
MPI.Finalize()
```

Launch four MPI ranks:

```
mpiexec -n 4 julia test.jl
```

Expected output:

```
Rank 0 using GPU CuDevice(0)
Rank 1 using GPU CuDevice(1)
Rank 2 using GPU CuDevice(2)
Rank 3 using GPU CuDevice(3)
```

H.6 Selecting GPUs

On a shared server, it is often desirable to restrict a program to a subset of the available GPUs. Before launching a computation, check current GPU usage:

```
nvidia-smi
```

You can restrict a program to specific GPUs using `CUDA_VISIBLE_DEVICES`. Use only GPU 0:

```
CUDA_VISIBLE_DEVICES=0 julia script.jl
```

Use GPUs 1 and 2:

```
CUDA_VISIBLE_DEVICES=1,2 julia script.jl
```

The selected GPUs are re-numbered internally. For example,

```
CUDA_VISIBLE_DEVICES=2 julia script.jl
```

makes physical GPU 2 appear as GPU 0 inside Julia. This is useful on shared servers where multiple users may be using GPUs simultaneously. When launching MPI jobs, the same variable can be used:

```
CUDA_VISIBLE_DEVICES=0,1,2,3 mpiexec -n 4 julia test.jl
```

Each MPI rank can then be assigned a GPU programmatically.

H.7 Checking Available GPUs

Before launching a large computation, check current GPU usage:

```
nvidia-smi
```

This allows you to identify which GPUs are currently in use.

H.8 Checking Which GPU Julia Is Using

Inside Julia:

```
using CUDA
CUDA.device()
```

I Multi-GPU Computing with Python

Many Python applications can use multiple GPUs simultaneously.

Typical examples include:

- PyTorch
- TensorFlow
- JAX
- MPI-based scientific computing

I.1 Loading Required Modules

Load the required software:

```
module load cuda/12.3.2
module load anaconda3/2024.02.1
module load openmpi/4.1.7-with-cuda
```

Verify:

```
module list
```

I.2 Installing Required Packages

If necessary:

```
pip install mpi4py
pip install torch
```

I.3 Testing CUDA

Create a file called `test-cuda.py`:

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.device_count())
```

Run:

```
python test_cuda.py
```

Expected output:

```
True
4
```

I.4 Verifying GPU Assignment

Create a file called `test-multigpu.py`:

```
from mpi4py import MPI
import torch
comm = MPI.COMM_WORLD
rank = comm.rank
torch.cuda.set_device(rank)
print( f"Rank {rank} " f"using GPU " f"
{torch.cuda.current_device()} " )
```

Run four MPI ranks:

```
mpiexec -n 4 python test-multigpu.py
```

I.5 Selecting GPUs

A program can be restricted to specific GPUs using `CUDA_VISIBLE_DEVICES`.

Use only GPU 0:

```
CUDA_VISIBLE_DEVICES=0 python script.py
```

Use GPUs 1 and 2:

```
CUDA_VISIBLE_DEVICES=1,2 python script.py
```

This is useful on shared servers where multiple users may be using GPUs simultaneously.

I.6 MPI with Python

Many scientific Python applications use MPI through `mpi4py`.

Install:

```
pip install mpi4py
```

Create a file called `test-mpi.py`:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD

print( f"Rank {comm.rank} " f"of {comm.size}" )
```

Run:

```
mpiexec -n 4 python test_mpi.py
```

Expected output:

```
Rank 0 of 4
Rank 1 of 4
Rank 2 of 4
Rank 3 of 4
```

I.7 Multi-GPU PyTorch

PyTorch supports multiple GPUs using Distributed Data Parallel (DDP).

Check visible GPUs:

```
import torch

print(torch.cuda.device_count())
```

A typical launch command is:

```
torchrun --nproc_per_node=4 train.py
```

where one process is launched for each GPU.

I.8 Troubleshooting

Verify that Python can see the GPUs:

```
nvidia-smi
```

Verify that PyTorch detects CUDA:

```
python -c "import torch; print(torch.cuda.is_available())"
```

Expected output:

```
True
```

J Multi-GPU Oceananigans on Nibi (Quick Start)

This appendix describes the minimal setup required to run Oceananigans benchmarks on Nibi using H100 GPUs and CUDA-aware MPI.

J.1 Login

```
ssh fpoulin@nibi.alliancecan.ca
```

J.2 Load Modules

Start from a clean environment:

```
module purge
module load StdEnv/2023
module load gcc/12.3
module load openmpi/4.1.5
module load cuda/12.6
module load julia/1.10.10
```

Verify:

```
module list julia --version
```

Expected:

```
julia version 1.10.10
```

J.3 Move the Julia Depot to Scratch

To avoid home-directory quota issues:

```
export JULIA_DEPOT_PATH=$SCRATCH/julia_depot

mkdir -p $JULIA_DEPOT_PATH
```

Verify:

```
echo $JULIA_DEPOT_PATH
```

J.4 Clone Oceananigans

Work in scratch rather than home:

```
cd $SCRATCH
mkdir -p software
cd software
git clone git@github.com:Clima/Oceananigans.jl.git
cd Oceananigans.jl
```

J.5 Instantiate Oceananigans

```
julia --project=.
```

Inside Julia:

```
using Pkg
Pkg.instantiate()
```

Exit Julia.

J.6 Instantiate the Benchmarking Environment

```
cd benchmarking
julia --project=.
```

Inside Julia:

```
using Pkg
Pkg.instantiate()
```

Exit Julia.

J.7 Request an Interactive H100 Node

Do not request only a GPU. Julia may be killed by the memory cgroup if too little system RAM is requested.

Recommended:

```
salloc \
--account=def-fpoulin \
--partition=gpubase_interac \
--gpus=h100:1 \
--cpus-per-task=8 \
--mem=32G \
--time=01:00:00
```

Larger memory requests (e.g. 64G) are recommended for benchmarking.

J.8 Verify the GPU

```
hostname
nvidia-smi
```

Example:

```
NVIDIA H100 80GB HBM3
```

J.9 Verify CUDA

Start Julia:

```
julia --project=.
```

Then run:

```
using CUDA
CUDA.functional()
CUDA.versioninfo()
```

Expected:

```
true
```

J.10 Configure MPI.jl to Use System OpenMPI

By default MPI.jl may install MPICH_jll. Configure MPI.jl to use the CUDA-aware OpenMPI installation provided by Nibi.

Inside Julia:

```
using MPIPreferences

MPIPreferences.use_system_binary(
    library_names=["libmpi"],
    extra_paths=[
        "/cvmfs/soft.computecanada.ca/easybuild/software/2023/" *
        "x86-64-v4/Compiler/gcc12/openmpi/4.1.5/lib"
    ])
```

Restart Julia.

Verify:

```
using MPI
MPI.versioninfo()
```

Expected:

```
binary: system
abi: OpenMPI
Open MPI v4.1.5
```

J.11 Verify CUDA-Aware MPI

```
using MPI
using CUDA
MPI.has_cuda()
```

Expected:

```
true
```

J.12 Simple MPI+CUDA Test

Create:

```
using MPI
using CUDA MPI.Init()

rank = MPI.Comm_rank(MPI.COMM_WORLD)

println("Rank = ", rank)
println("Device = ", CUDA.device())
println("CUDA aware = ", MPI.has_cuda())

MPI.Finalize()
```

Run:

```
mpiexec -n 1 julia --project=. test_gpu_mpi.jl
```

Expected:

```
Rank = 0
Device = CuDevice(0)
CUDA aware = true
```

J.13 Run a Benchmark

Move into the benchmarking directory:

```
cd benchmarking
```

Example benchmark:

```
julia --project=. run_benchmarks.jl \
--size=720x360x100 \
--device=GPU
```

This benchmark uses approximately

6.5 GiB

of GPU memory on an H100 and achieves approximately

1.94e8 grid points/second

for the Earth-Ocean benchmark.

J.14 Notes

- Use scratch for all Oceananigans work.
- Use scratch for the Julia depot.
- Nibi provides CUDA-aware OpenMPI; rebuilding MPI is not necessary.
- Always launch MPI-enabled scripts with:

```
mpiexec -n N julia --project=. script.jl
```

- Always specify memory in GPU allocations.

K Contributing to This Handbook

This handbook is maintained through a public GitHub repository.

Suggestions, corrections, and improvements are welcome.

K.1 Repository

The source files for this handbook are hosted on GitHub:

`https://github.com/francispoulin/uwaterloo-fluids-computing-handbook`

K.2 Reporting Errors

If you discover:

- incorrect commands;
- broken examples;
- outdated software versions;
- typographical errors;
- unclear instructions;

please open a GitHub Issue.

Describe:

- the section containing the issue;
- the command that was executed;
- any error message;
- a proposed correction (if known).

K.3 Suggesting Improvements

Suggestions for new sections, tutorials, appendices, and examples are encouraged. Examples:

- new software packages;
- updated Julia workflows;
- Alliance cluster instructions;
- additional GPU examples.

K.4 Submitting Changes

Users familiar with Git may submit a pull request.

All contributions are reviewed before being merged.

K.5 Source Files

Do not edit the PDF directly.

All modifications should be made to the source files contained in the repository.